P395: Guide 9: Neural Networks

For this week, I have prepared a notebook that contains the lines of code needed to set up a neural network ("neural net"). Your task will be to read in the data, organize it, and then train and test the neural net.

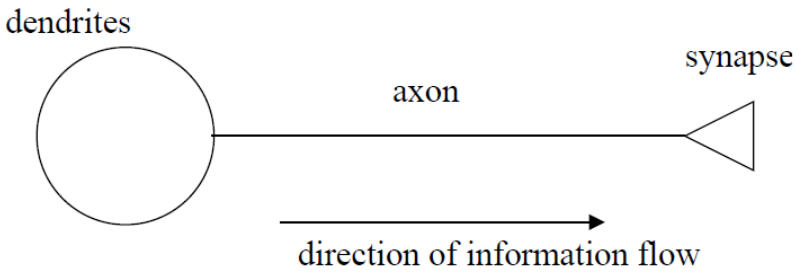
1. Download from Canvas the notebook 'neural_net_template.ipynb'. Rename it to 'Guide9_your_name'. You may find it necessary to put in more helpful headings, etc., to keep your notebook organized.

Please read through all '**' material before you start trying to code/make modifications.

This week we are going to look at artificial neural networks (ANNs) for learning complex relationships. For many tasks where a system can be well modeled/described by a set of mathematical equations, there is no need for ANNs (despite everyone wanting to apply them to everything). However, for systems where models are unknown, ANNs have been shown to be able to learn relationships and even be predictive. This week, we will look at using an ANN to classify a phase transition. This activity will just be scratching the surface of all that is possible with ANNs. There are many excellent tutorials on the internet that walk you through how to code up and train ANNs if you simply search for the problem you are trying to solve.

**Background: Neurons and Neural Networks

ANNs are inspired by the biology of real neural networks. Our brain is composed of roughly 10^{11} neurons. A single neuron cell acts as an information-processing unit (see simplified schematic below). It i) receives information (in the form of chemical signals from input neurons connected to its dendrites), it ii) relays information (via electrical pulses known as action potentials that propagate down the neuron's axon), and it iii) sends information (in the form of chemical signals) to other neurons through synapses.



The strength of connections between neurons is mediated by synapses and these can change with time as learning occurs, thus altering how information is processed. Thus, one of the ways neural networks 'learn' is by adjusting the connections/weights between neurons (altering the overall activity of the network) as pattern associations are made which (e.g., see Hebbian learning that was proposed by McGill psychologist Donald Hebb in 1949). The activity of the network can then be used as output to drive other processes.

**Action Potentials and Neuron Firing

The physics of how neurons function is fascinating, and we could do a whole activity (or two) on modelling the behaviour of a single neuron. In short, there are ion channels in a neuron cell's membrane that cause currents of ions to flow across the cell. If there is a current, then there must be an electrical potential difference across the cell. What's amazing is that some of these channels open or close in response to the electrical potential. This provides feedback and can trigger the cell to generate a potential pulse (known as an action potential) that can propagate down the axon. This electrical signal can then stimulate chemical processes at the end of the neuron at its synapses. In the 1950's, Hodgkin and Huxley came up with their now-famous model consisting of a set of coupled differential equations to describe the physics and chemistry of how the action potential is formed. Their model is incredibly successful at capturing much of the dynamics of action potentials in terms of the physics of the ion channels and cell membrane.

A simpler model that describes the firing of a neuron is the following: Imagine the *i*th neuron of our neural network has a set of input neurons, coupled to it by synapses, with weights w_{ij} for the jth input. Each of these input neurons has a firing rate x_j quantifying how many action potentials they send per unit time. The input signal from one of these neurons is thus $w_{ij}x_j$. The total input signal received by neuron *i* is then $I_i = \sum_j w_{ij}x_j$. To capture the nonlinear physics of the action potential, one models the firing of the receiving neuron as a nonlinear function of the total input signal: $x_i = f_i(I_i)$. This could be a simple threshold function (e.g., a Heaviside function) or some sigmoid-type function (e.g., tanh or the logistic function) so that the output firing rate is bounded. The resulting dynamics of the network and what it can learn are primarily specified by the weights w_{ij}.

**Artificial Neural Networks

The above simplified model of a neuron's firing forms the basis of ANNs. An ANN consists of a set of *N* 'neurons' that each have a certain firing rate x_j. The architecture of the network is completely described by the set of weights w_{ij} that connect pairs of neurons together. The output of the *i*th neuron is

$$x_i = f\left(\sum_j w_{ij}x_j + b_i\right)$$

For the ANNs that we will construct, the nonlinear function will be the 'ReLU' = Rectified Linear Unit function, $f(x) = \max(0,x)$, which zeroes all negative inputs but otherwise gives a linear output (note it is still a nonlinear function). The need for nonlinearity is key. A bunch of linear functions is still linear and turns out to be quite limited in what it can learn. The amazing thing about neural nets is the "Universal Approximation Theorem", which says that a neural network with just one hidden layer (see below) can approximate any function as accurately as you want. We will exploit this property in this activity to learn a couple complex relationships.

We will look at one type of architecture that forms the basis of solving a variety of problems. It is a feed-forward network (shown below) which only has connections between layers and not within (this

helps to speed its training). The key to solving a given problem with an ANN is setting the architecture (i.e., the layout of the nonzero weights w_{ij}) and then learning through training the values of the w_{ij} and the biases b_i. Much of the foundational work for the modern methods we now use to train feed-forward networks to solve a variety of learning tasks was carried out by Geoffrey Hinton's group at the University of Toronto in the 1980s. It is only in recent years that enough computational power has become available to implement these methods on a scale to solve real-world problems.

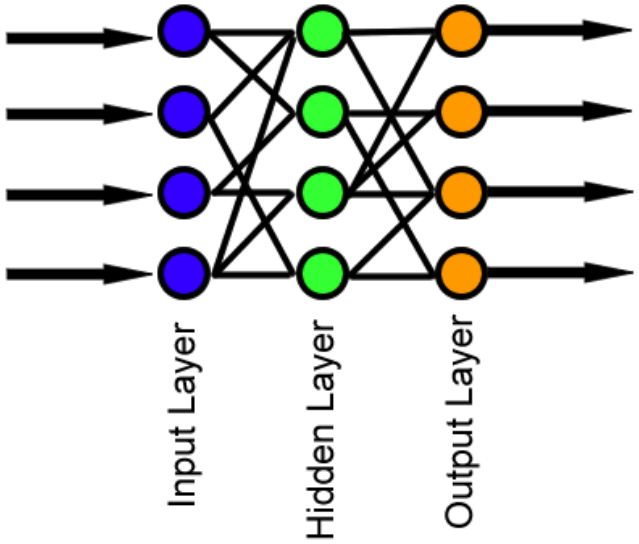


Figure: Schematic of a feed-forward neural network (courtesy Wikipedia images).

**Wide versus Deep Neural Networks

Another thing to consider when building an ANN is choosing how many neurons to put into it. For the case of feed-forward networks, this involves choosing both the width of a layer (i.e., how many neurons in a layer) and then how many layers to use. One approach is to solve the problem using a wide, dense neural network, with only a few layers. Because of the Universal Approximation Theorem, such networks can approximate any functional relationship between input and output. Their problem is that they have a vast number of trainable weights and are not very interpretable (i.e., they are just a black box). Another approach is convolutional neural networks (CNNs) that use many narrow layers of convolutional filters to learn important patterns that relate input to output. The low-lying layers find coarse features, and the higher layers build up a grammar from them. The result is that the network learns the important feature set for solving a problem (much like our visual system). These networks often need fewer trainable parameters and are interpretable in that one can look at the learned features that are being extracted from the inputs. They become 'deep' when there are many convolutional layers stacked on top of each other, hence the term 'deep learning'.

** Machine Learning, Optimization, and Overfitting

Machine learning is a fancy way of saying, 'fitting a high-dimensional model to data'. (It is slightly more than that, but that is the basic premise.) We have a set of inputs (there could be many such variables [called features]) and a set of observed outputs (this also could be many variables). We seek to fit some model (with many fittable parameters) to the data, and we would like it to generalize. This means that we don't want it to overfit our data so that it does poorly on data it has never seen before. To minimize overfitting, the data is split into training, validation, and test sets. The test set is left out of the fitting process. The training and validation sets are used to 'learn' the parameters in the model. An iterative procedure is used to update the parameters in the model using the training data, and then the trained model is scored on the validation data. This process of training and scoring on the validation data is continued until the scores on the validation data consistently get worse. This is shown in the figure below where the quality of the fit on the training set (shown in blue) always improves. However the quality of the fit on the validation set (red curve) eventually starts getting worse. When this happens, it signals that one is moving into the overfitting stage (i.e., your model is losing its generality and is now just trying to fit the 'noise' in the training data). This provides a criterion for stopping the fit (known as *early stopping*) of your complex model and is what we will use when fitting the feed-forward neural networks.

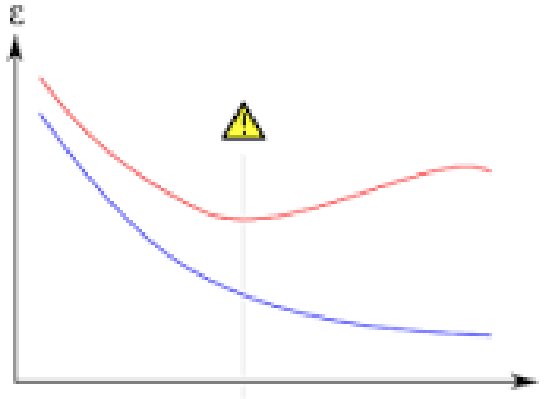


Figure The computed loss as a function of training cycles (epochs) on a training set (blue curve) and validation set (red curve). The epoch where the loss on the validation data starts to consistently go up (labelled by a warning symbol) is a plausible point to stop training to minimize overfitting. (Courtesy Wikipedia images).

**Python and Neural Nets

There are several packages in Python for setting up and training neural networks. As of writing, two popular choices are TensorFlow (https://tensorflow.org/) and PyTorch (https://pytorch.org/). We will use TensorFlow on the backend with the Keras API. PyTorch offers greater flexibility, but has a more complex syntax to learn. Keras is simply a set of wrapper functions that make setting up a neural net relatively easy, compared to using the more complex and lengthy syntax of the TensorFlow backend.

The Problem: Image Classification and Predicting a Phase Transition

One class of problems particularly well suited to neural nets is image classification. Given an image of an object that belongs to one of a set of categories, the goal is to predict which category the object belongs to. Our visual system is excellent at doing this by learning important patterns/features in images, starting from simple ones like edges, that then get built into a larger grammar of textures. When a given object is presented, a subset of these features will 'fire' that then activate output neurons which encode the given category. We can construct an ANN that has an architecture inspired by our visual system and essentially accomplishes this.

Task 1: Reading in and preparing the data

In this activity, your task is to classify snapshots of a 2D system of spins as being above or below the critical temperature. Here, the input is a 2D image of binary spins, and the output is just a single number representing the likelihood of the image being below the critical temperature (i.e., if the output ≈ 1 it is below the critical temperature, while an output ≈ 0 would be above). If we know the temperature that the image was taken at, and we know the critical temperature, we can assign a binary label to all our images (1 = at or below critical temperature, and 0 = above critical temperature). This forms our set of input/output pairs that we will use to train our classifier. We will use a neural net to do the classification.

1. Download the file 'Ising.zip' from Canvas and upload to your Guide9 folder in syzygy. Then open a new terminal in syzygy (blue + sign in top-left corner and scroll down to Terminal) and navigate to your Guide9 folder. Run the command `unzip Ising.zip` to extract the archive. This will create a folder named Ising. In it are subdirectories labeled by their temperature. Within each of these are 500 files of 2D arrays that represent spin configurations.
2. In your notebook, fill in the code to read in all the image files from the directories into a single array of inputs *X*. How many images in total are there? Each image is *L*x*L* pixels, what is *L*? Reshape each element of *X* to be of size (*L*,*L*,1) using the `np.reshape` command. This is because we are treating them as images, and in Keras you need to specify (in the third axis) how many channels your image has (e.g., an RGB image would have 3 channels); in this case the number is 1. As you read in the images, you will also need to create an array *Y* of output labels. For each image, assign a label based on its temperature (1 = at or below T_c and 0 if it is above). These images were simulated using the 2D Ising model on a square lattice so $T_c = 2.27$ (in the appropriate units). You should also have an array that records the temperature *T* of each image, that you will use later. Your set of *X* and *Y* form your dataset that you will use for training, validation, and testing.
3. Visualize some of the images at temperatures i) well below T_c, ii) near T_c, and iii) well above T_c. What about the images is changing as the temperature changes?
4. Fill in the missing steps in the notebook's code to split the data into training, validation, and test sets. The important step here is shuffling your data before you split, so that all temperatures are roughly equally represented in each set.
5. Go to the section 'Dense Neural Network' (DNN) to see the code for defining a simple neural network.

Task 2: Training a Dense Neural Network

You are now looking at some code that sets up an ANN. (I am using the functional API of Keras, as I find it provides a bit more flexibility [and readability] than using their Sequential syntax). All neural nets start with an `Input` layer that specifies the shape of the input data. In our case it is a 2D image, with shape (*L*,*L*,1). After the input layer, the rest of the layers are up to you and form the neural net. The last layer is the output layer; in our case, we are doing binary classification, so it is just a single neuron (if we had more categories, then there would be more output neurons). We want its output to reflect a probability, so we will use a sigmoid function whose output goes between 0 and 1.

With our architecture defined, we wrap it up into a Keras `Model`, that specifies the input layers and output layers. We then `compile` our network for fitting, which means we must choose the type of loss function and the optimizer that we will use to solve it. We will use 'stochastic gradient descent' ('SGD') and play with its learning parameter (see the end of the activity for an exercise on SGD). There are some better solvers (e.g., Adam) that can train faster, but SGD isn't too bad. Our loss function for a binary classification problem is binary cross-entropy, defined as

$$l = \sum_k [-y_k \log(p_k) - (1 - y_k) \log(1 - p_k)]$$

where y_k is the label for the *k*th image and p_k is the output from the neural net. You can see that this loss is minimized when $p_k \approx y_k$. We can also specify additional metrics to measure performance, and since we are doing classification, 'accuracy' is a good one to monitor (i.e., how well are we doing at predicting the correct label).

The last thing we specify is our early-stopping criterion for when to stop fitting. The key argument here is `patience`, which specifies how many epochs to allow the validation loss to increase before stopping because we are starting to overfit. We'll go with a patience of 3 epochs.

Dense Neural Network Architecture

DNNs simply consist of one layer of neurons after the other, and they are typically wide, meaning that a given layer usually has 100s to 1000s of neurons. In Keras, this is specified with the `Dense` layer, where the argument specifies the number of neurons. The input layer forms the first layer that has as many neurons as there are input variables. Since our input is 2D and we want to feed it into a 1D layer of neurons, we need to `Flatten` the input, by simply flattening it into a 1D vector. This can then be connected by a set of weights to the next dense layer and so on. The output of a dense layer is just a linear combination of the inputs using the weights (as in our equation above in the section on neural networks). We then need to pass this to a nonlinear function, specified by the `Activation` layer; a popular choice is "ReLU". Another common is to "dropout" neurons during the training so that the DNN is forced to learn several ways to solve the problem. This is accomplished with a `Dropout` layer where the number represents the proportion of neurons to randomly drop at each cycle of training. This DNN has 2 wide layers, each with dropout, and ends in a single sigmoid output neuron.

1. Run the cell containing the DNN network and see the summary. How many fittable parameters does it have?

Now we need to train the network. This is done in the next cell using the `fit` function of the model. The first arguments are the input/output training data. Then we specify the validation data to be scored as the model is fit to guard against overfitting. The number of training cycles called an epoch (this corresponds to one loop over all the training examples) needs to be given. For the optimizer we also need to specify how many samples we want to use in a mini-batch for calculating the updates to the weights. If it is size = 1, then only one sample is used at a time. This is pure stochastic gradient descent and will run slow. A batch size around 32 is usually good.

2. Run the `fit` cell. If your data is organized properly, you should see it fitting the data giving you info on how the fit is performing. The fit will either run for the number of epochs or stop if the early stopping criterion is met. If you run the full number of epochs, re-run the cell until the early-stopping criterion stops the fit.
3. Now use your fitted network to make predictions on your test set. Run the cell with the `evaluate` function. What is the accuracy and loss on your test set?
4. Run the cell that calls the `predict` function of the model. The prediction yields a set of probabilities p_k of being below the critical temperature for each image in your test set.
5. Calculate the average probability $< p_k >$ as a function of the temperature of the test images.
6. Plot the average probability vs. temperature. Given your plot, has the trained NN been able to discover the critical temperature (i.e., where $< p_k > = 0.5$) ?

Task 3: Tweaking the DNN

Adjusting learning rate

Go back to where we defined your network. There was a learning rate. If you make it smaller, the convergence will be slower, but it may also be less prone to random noise in the stochastic gradient descent.

1. Set `learning_rate=0.001` and refit your ANN. Can you get a better accuracy on your test set?

Tuning network parameters

Now the fun begins. Is this the best architecture? There are lots of potential ways to change the network to improve the accuracy. We could i) add more layers, ii) change the width of the layers, iii) change the dropout rate, and iv) change the activation function. We'll just do two.

2. Add another identical dense/ReLu/dropout layer block, so that there are now 3 dense layer blocks in the network. Refit the model and make predictions on the test set. Did the accuracy improve?
3. Try a couple different dropout values, say 0.1 and 0.3. What do you notice about overfitting now? Does the fitting stop sooner with these lower dropout rates? And did the accuracy on the test set improve?

Task 4: Augmenting the training dataset

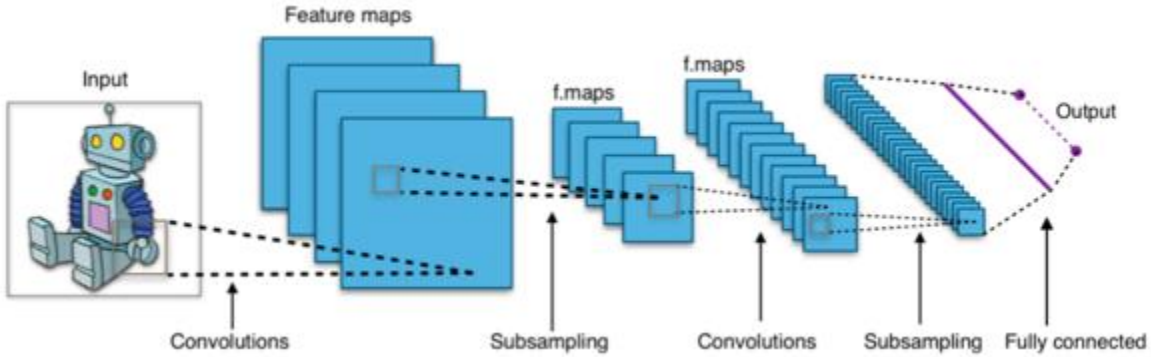
It can be time-consuming and frustrating making these tweaks to ANNs, as the gain in improvement is usually pretty marginal. The biggest way to improve things is to get more training data. If your system

has symmetries, one can usually then augment the dataset using the data you already have. In our case, a 2D spin configuration can be flipped vertically or horizontally, and both would also be perfectly acceptable configurations. By doing these 2 symmetry operations, we triple the size of our training dataset.

1. Go back to the part of the notebook entitled 'Augmenting the data'. See how the 2 flip operations are preformed. Run the cell to triple the number of examples in your training set. Go back and refit the original network using the augmented training data. What is the improvement in accuracy on the test data?

Task 5: Training a Convolutional Neural Network (CNN)

The DNN we trained above is basically a black box. There are methods that can figure out the importance of various parts of the input that lead to the output given just the black box; these help in the *interpretability* of the model learned by the ANN. But what if we built an ANN that had an architecture that itself was interpretable? Convolutional neural nets (CNNs), described in the introductory material, can accomplish this. Each layer in a CNN has a collection of 'filters' that learn features present in the image (see figure below). These filters are little 2D image patches that are convolved over the image. The key next step is to compress (or subsample) the information by only



taking the maximum value of the convolution in a patch – this gets rid of fluctuations and the importance of location (i.e., for identifying whether a picture contains a robot, it shouldn't matter where in the image the robot is located). This process is repeated with another layer of filters, except these filters convolve the previous outputs (which are convolutions); this layer learns the grammar/texture of the simple low-lying features. Again, subsampling shrinks the size of the resulting output. The last step is to take the maximum signal of the last set of convolutions to represent the final summary signal of features. Thus, these convolutional layers have extracted a set of abstract features that best categorize the image. This feature set is passed into a fully connected dense layer that then learns the logic operation that does the classification.

1. Go to the section entitled 'Convolutional Neural Network'. Let's work through its architecture.

After the `Input` layer, there is a 2D convolutional layer (`Conv2D`). (Note, here we don't flatten our input image.) This applies filters that are 2D matrices of a given size (here 2 x 2) to the image. These 2D matrices get 'strided' over the image, giving a score at each position that intuitively is just how well that patch of image matches the given filter. There are 32 filters – meaning there are 32 low-level features that will be learned. It then applies the 'RelU' function. Then the outputs are subsampled using a

maximum pooling layer (`MaxPooling2d`) which in this case takes the top score over a 2x2 patch, thus cutting the original image down by 1/4. This process is repeated several times, where we add more filters to learn more complex combinations of the lower-level features present in the images. Finally, after one final subsample, the features are flattened into a vector that provides the final set of extracted features. This is then passed to a dense layer (`Dense`) whose output is then fed into the single last neuron that has a sigmoid activation.

2. Run the cell with the CNN architecture and look at the number of parameters it has compared to the DNN.
3. Now `fit` this model to the same (augmented) training data used to train the DNN. Look at the accuracy on the test data and again plot the average probability versus temperature on the test set. Does it do better or the same as the DNN?

Again we could change parameters such as the number of Conv2D layers, the number of filters, the size of the dense layer, etc. to see if we can improve accuracy. But let's move on to predicting on data from a model that is not an Ising model.

Task 6: Predicting on novel data

The neural net does a surprisingly good job of being able to categorize whether a configuration of spins is above or below the critical temperature. What if we gave it a spin configuration from a model that has a phase transition, but came from a different microscopic model? Can the trained neural net predict which configurations are above or below the unknown critical temperature?

1. Download the `nonIsing.zip` from Canvas and carry out the same procedure on syzygy to extract its contents. You will get a directory called nonIsing. It is organized in the same way as the data for the Ising case. Create a set of inputs *X* from this data (no need to augment it here, since you are not training). Note we don't know the *Y*'s in this case.
2. Use the `predict` method of your model on this set of data. Plot the average probability versus temperature as you did above. What would you predict is the likely critical temperature for this system?